

Robust Mobile Manipulator Control via Deep Reinforcement Learning with IMU-Based Disturbance Compensation

Marco Reis

marcoreis@alunos.utfpr.edu.br

Prologue

Initial robotic system: **Pioneer 3-AT** + **Movemaster RV-M2**

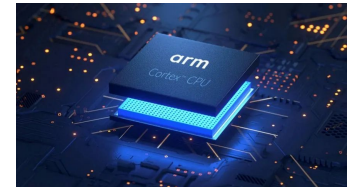
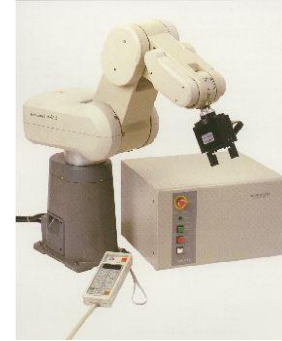
MacOSX Environment

Arm Platform

ROS Noetic

Simulation

Q-learning



Introduction

Initial search (references)

```
@inproceedings{xinlan2023trajectory,  
  title={Trajectory Planning and Obstacle Avoidance  
Behavior of Manipulator Based on Q-learning  
Algorithm},  
  author={Xinlan, Guo and Tao, Li and Jian, Zhang},  
  booktitle={2023 IEEE International Conference on  
Control, Electronics and Computer Technology  
(ICCECT)},  
  pages={1235--1241},  
  year={2023},  
  organization={IEEE}  
}
```

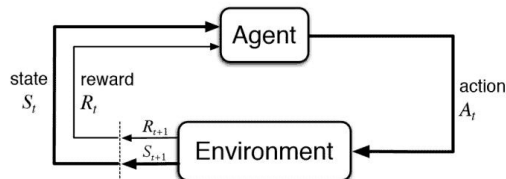
```
@article{yu2025trajectory,  
  title={Trajectory tracking control for robotic  
manipulator with disturbances: a double-Q  
reinforcement learning method},  
  author={Yu, Dehai and Sun, Weiwei and Li, Yongshu  
and Luan, Zhuangzhuang and Zhang, Zhongcai},  
  journal={Applied Intelligence},  
  volume={55},  
  number={11},  
  pages={818},  
  year={2025},  
  publisher={Springer}  
}
```

Q-learning

Reinforcement Learning

The central idea of Q-learning is to train an "agent" to learn the best sequence of actions to take in a given "state" to maximize the total "reward" in the long run. It does this by estimating the quality value, or Q-value, of each (state-action) pair.

NO previous model.



Equação de Atualização (baseada na Equação de Bellman)

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Onde:

- $Q(s, a)$: O Q-valor atual para o estado s e ação a .
- α (Taxa de Aprendizagem): Controla o quanto a nova informação anula a antiga. ($\alpha \in [0, 1]$).
- R : A recompensa imediata recebida ao tomar a ação a e passar para s' .
- γ (Fator de Desconto): Controla a importância das recompensas futuras. Um γ próximo de 0 faz com que o agente considere apenas as recompensas imediatas; um γ próximo de 1 o faz buscar recompensas de longo prazo. ($\gamma \in [0, 1]$).
- $\max_{a'} Q(s', a')$: O valor Q máximo que pode ser obtido do próximo estado s' (ou seja, a melhor ação a partir do próximo estado).

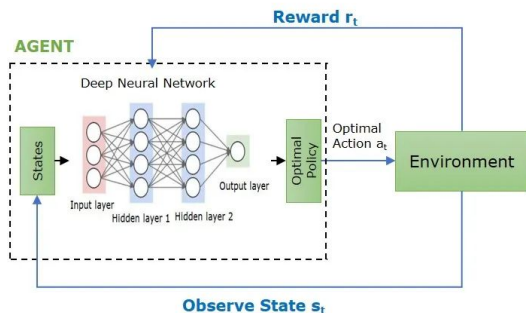
Deep-Q-Learning

Q-Learning evolution

Replacement of the Q-Table
with a Deep Neural Network.

Replay Buffer

Target Network



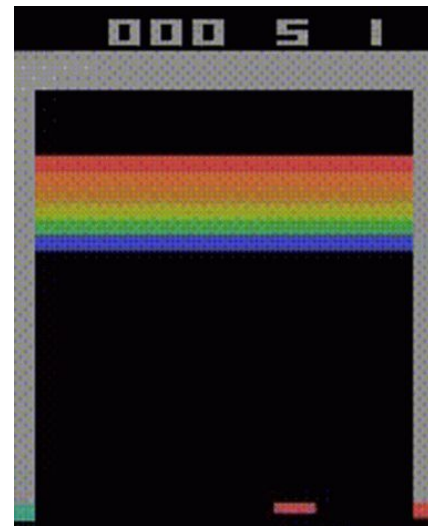
Loss function

$$\text{Loss} = \mathbb{E} \left[\left(\underbrace{R + \gamma \max_{a'} Q(s', a', \theta_{\text{alvo}})}_{\text{Alvo Q (calculado pela Target Network)}} - \underbrace{Q(s, a, \theta)}_{\text{Previsão Q (calculada pela Rede Q)}} \right)^2 \right]$$

Onde:

- Onde:
- O termo da esquerda é o **Alvo Q**, com **pesos fixos** (θ_{alvo}).
- O termo da direita é a **Previsão Q**, com **pesos treináveis** (θ).

Ao minimizar essa perda, a Rede Q aprende a se aproximar do Alvo Q estável, resultando em um aprendizado muito mais eficaz e poderoso.



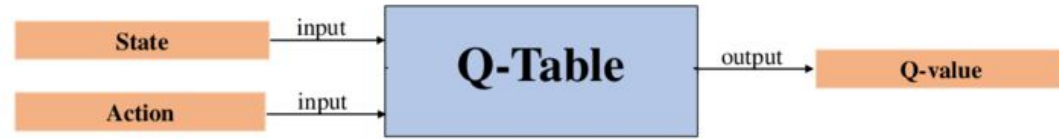
2015

Q-Learning vs. Deep-Q-Learning

Central point: **agent's brain**.

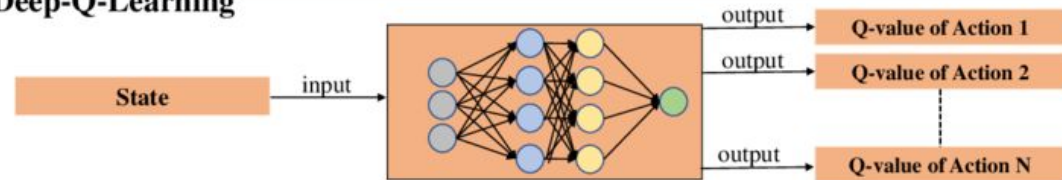
In Q-learning, the Q-table serves as the agent's brain, where the agent's state and action are the inputs of the Q-table, and the Q-value of these inputs is the Q-table's output.

Q-Learning



In Deep-Q-Learning, a deep neural network acts as the brain, where the state is the input of the deep neural network, and the approximation of all Q-values of each possible action corresponding to that specific state is the neural network's output.

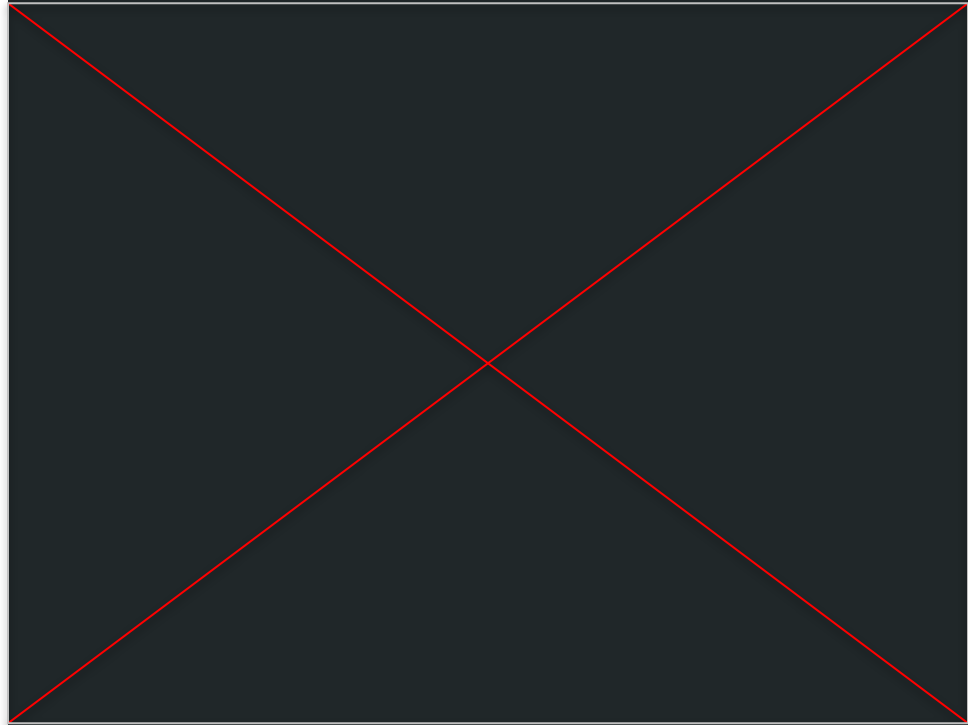
Deep-Q-Learning



The problem

The challenge

The solution

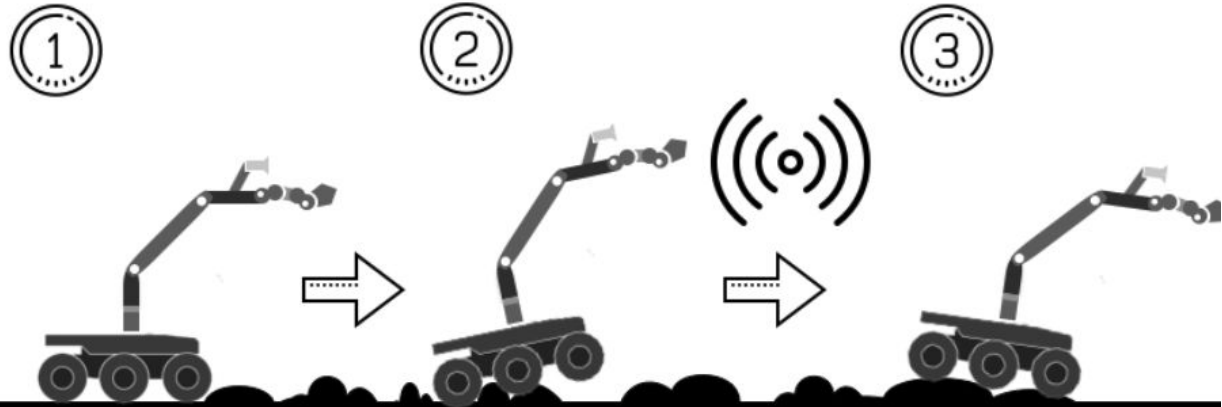


Challenges

To execute precise end-effector movements despite disturbance.



Carry out missions as efficiently as possible.

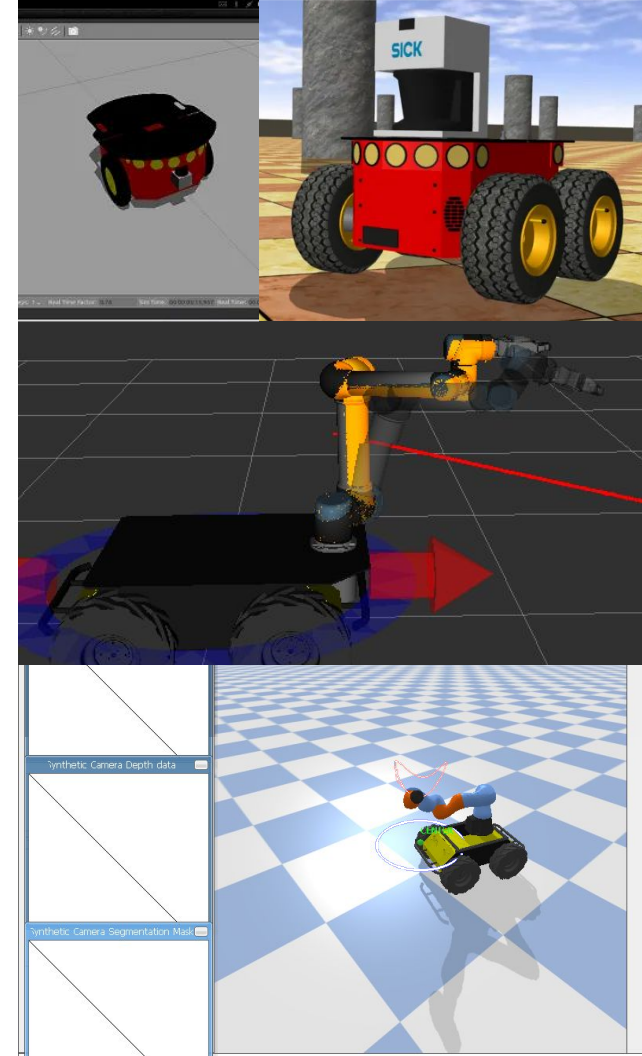


The dataset (available/unavailable)

How to develop a
project if there is no
dataset?

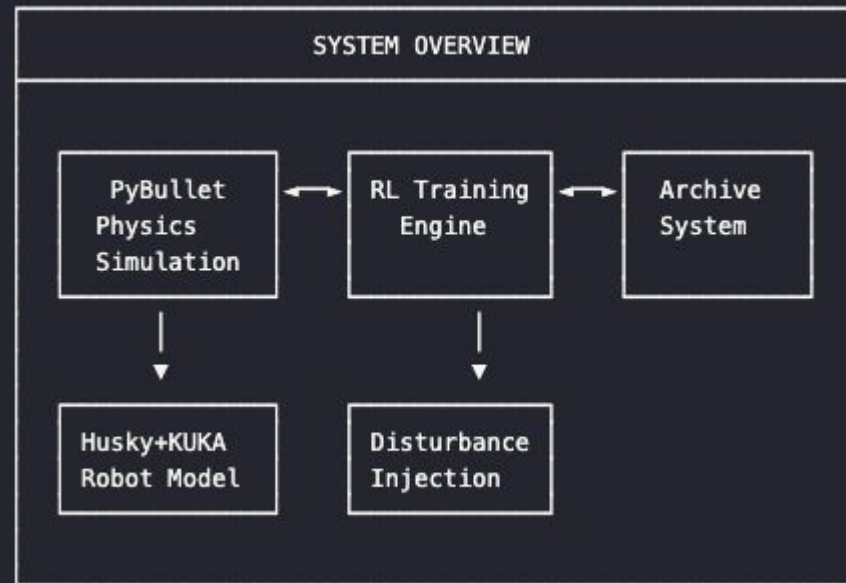
talking about the simulation

- urdf distinct from the mobile platform and the manipulator
- necessary to join the two devices
- establish the appropriate physics
- emulate different disturbances, in addition to the intensity level and its direction



Experiment Methodology

strong  Python
Pybullet simulation



Husky-Kuka RL System

Hardware specification

Husky Base: 4-wheel drive, differential steering

KUKA Arm: 7-DOF, iiwa14 model

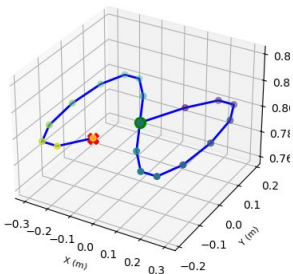
End-effector: Parallel gripper

Sensors: Virtual IMU (linear/angular velocity)

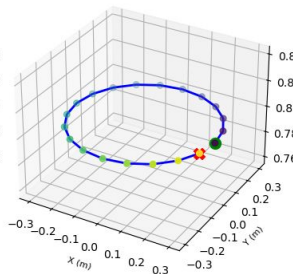
Workspace: $1.5\text{m} \times 1.5\text{m} \times 1.2\text{m}$ (X×Y×Z)

Trajectories

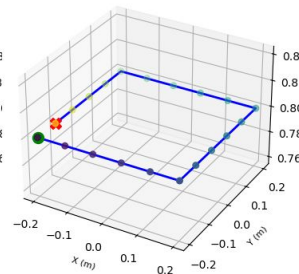
Figure-8



Circle



Square



Key Capabilities

- ✓ Autonomous Trajectory Following: Smooth circular, Figure-8, and custom paths
- ✓ Disturbance Robustness: 5 scenarios from gentle to extreme
- ✓ Real-time Learning: DQN with GPU acceleration (MPS/CUDA)
- ✓ Performance Analytics: Component-wise accuracy tracking (X,Y,Z)
- ✓ Data Preservation: Automatic archiving of all training data

Husky-Kuka RL System

Five disturbance scenarios

Scenario	Force Range	Frequency	Real-World Analogy
 none	0N	Never	Laboratory conditions
 random	$\pm 50\text{N}$	Every step	Bumpy terrain, collisions
 periodic	$\pm 100\text{N}$	Every 50 steps	Machinery vibrations
 continuous	$\pm 10\text{N}$	Every step	Wind force, slope gravity
 impulse	$\pm 200\text{N}$	Single shock	Emergency stop, collision

Intensity Level

1. none
2. normal
3. golden ratio

Deep Q-Network (approach)

Architecture:

Neural Network: 35 inputs $\rightarrow$ 128 $\rightarrow$ 128 $\rightarrow$ 35 outputs
Device: GPU (MPS/CUDA) or CPU fallback
Memory: 10,000 experience buffer
Batch Size: 32 samples per update
Target Network: Updated every 100 steps

Hyperparameters:

- Learning Rate: 0.001
- Discount Factor: 0.99
- Exploration: $\epsilon = 1.0 \rightarrow 0.01$ (decay)
- Training: Experience replay with random sampling

Husky-Kuka RL System

Q-Learning (approach)

Architecture:

```
Q-Table: Dictionary {state: [Q-values]}  
State Discretization: 2 decimal places  
Memory: Grows with explored states  
Updates: Direct Q-value modification
```

Hyperparameters:

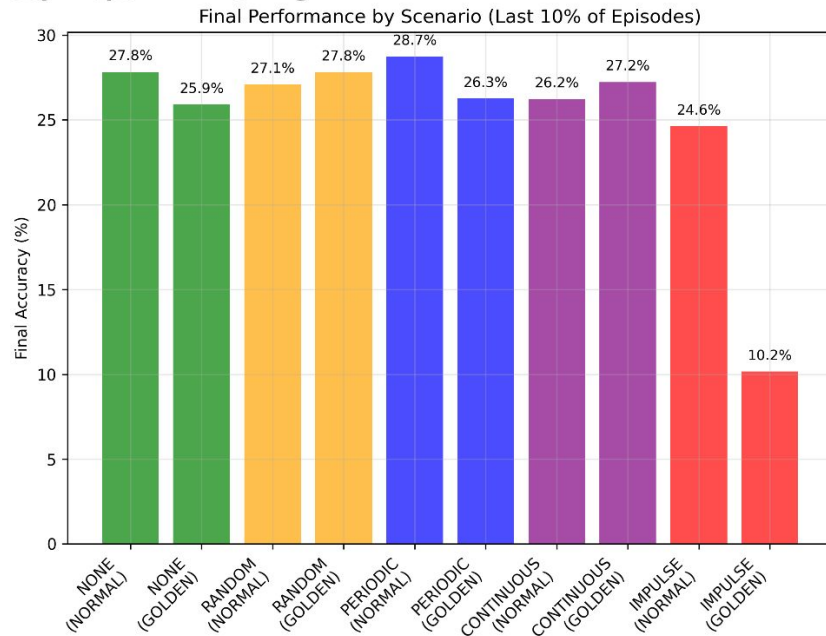
- Learning Rate: 0.1
- Discount Factor: 0.99
- Exploration: $\epsilon = 0.2 \rightarrow 0.01$ (decay)
- Training: Immediate Q-value updates



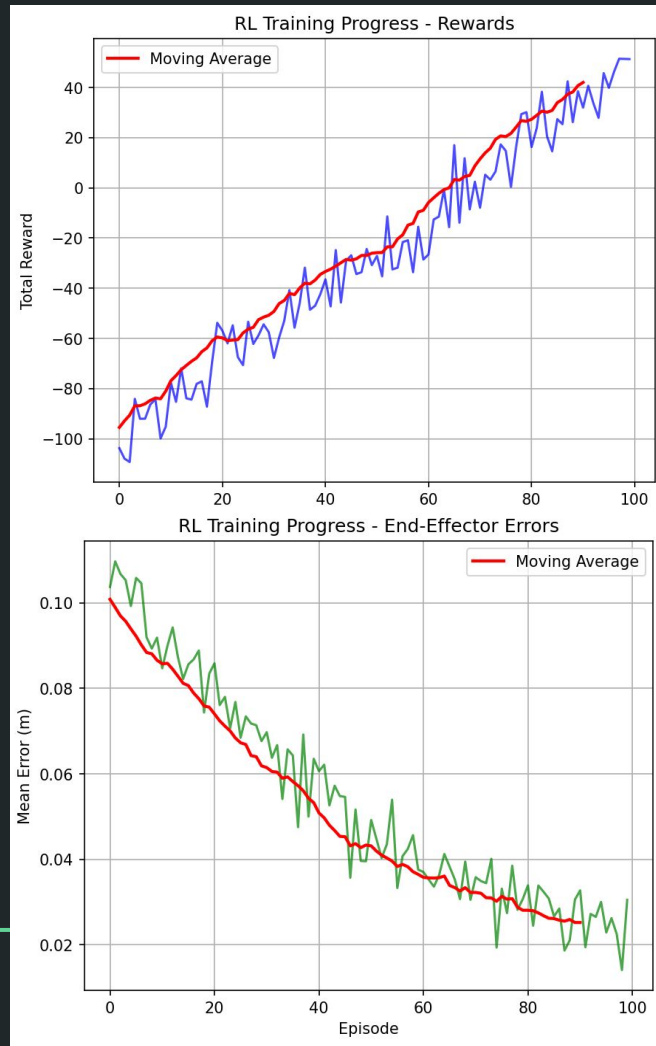
Aha!

The results

- Initial overview
 - Total Episodes Analyzed:
 - 5,000 episodes across 10 scenario combinations
 - Best Performing:
 - RANDOM_NORMAL (27.6% success rate)
 - Most Challenging:
 - IMPULSE scenarios (21.8% average success rate)
 - Scenarios:
 - NONE, RANDOM, PERIODIC, CONTINUOUS, IMPULSE
 - Intensities:
 - NORMAL and GOLDEN (Golden Ratio) force multipliers

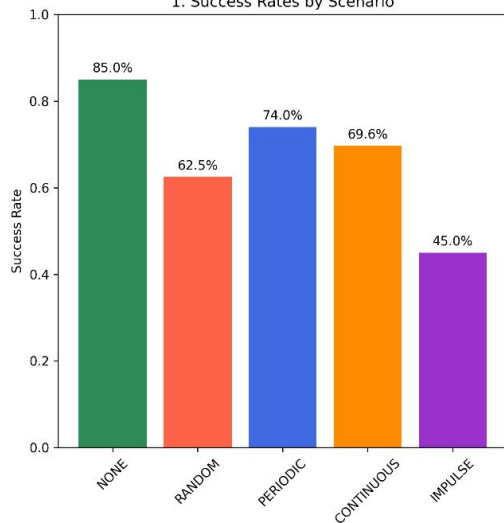


RL Training

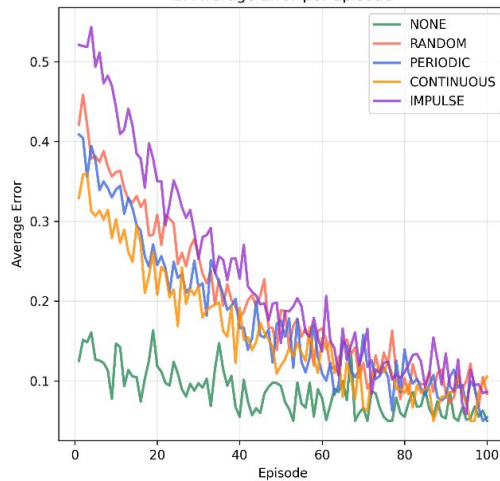


RL Training Analysis Dashboard

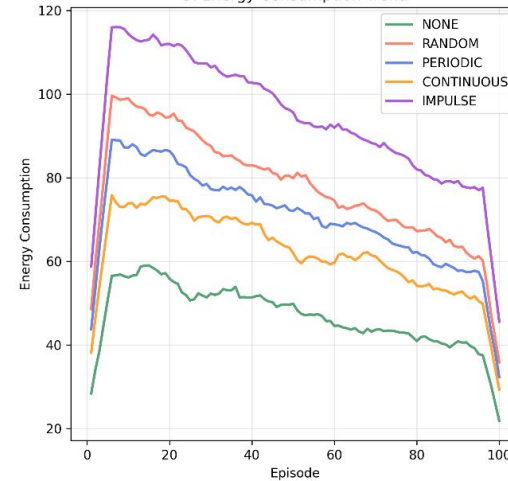
1. Success Rates by Scenario



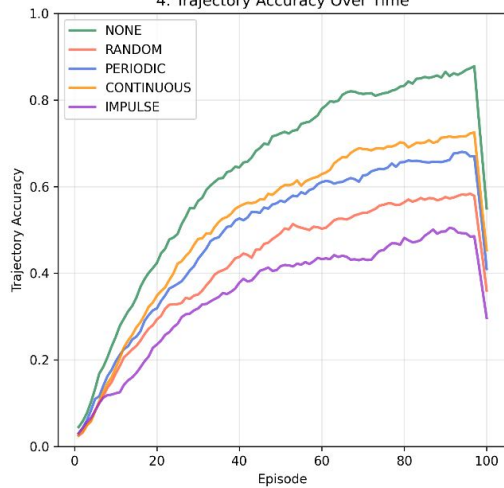
2. Average Error per Episode



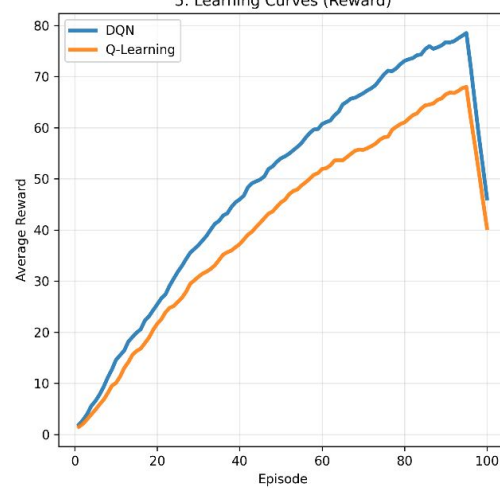
3. Energy Consumption Trend



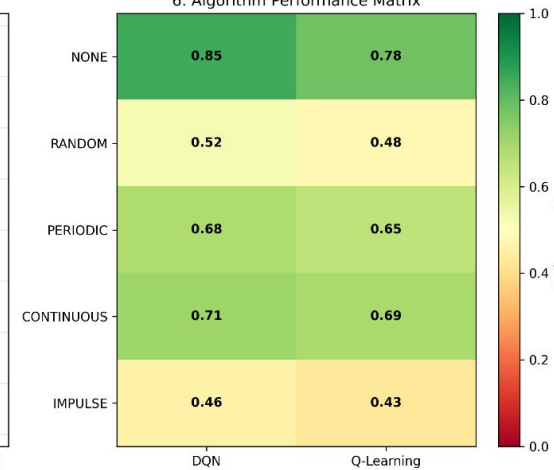
4. Trajectory Accuracy Over Time



5. Learning Curves (Reward)



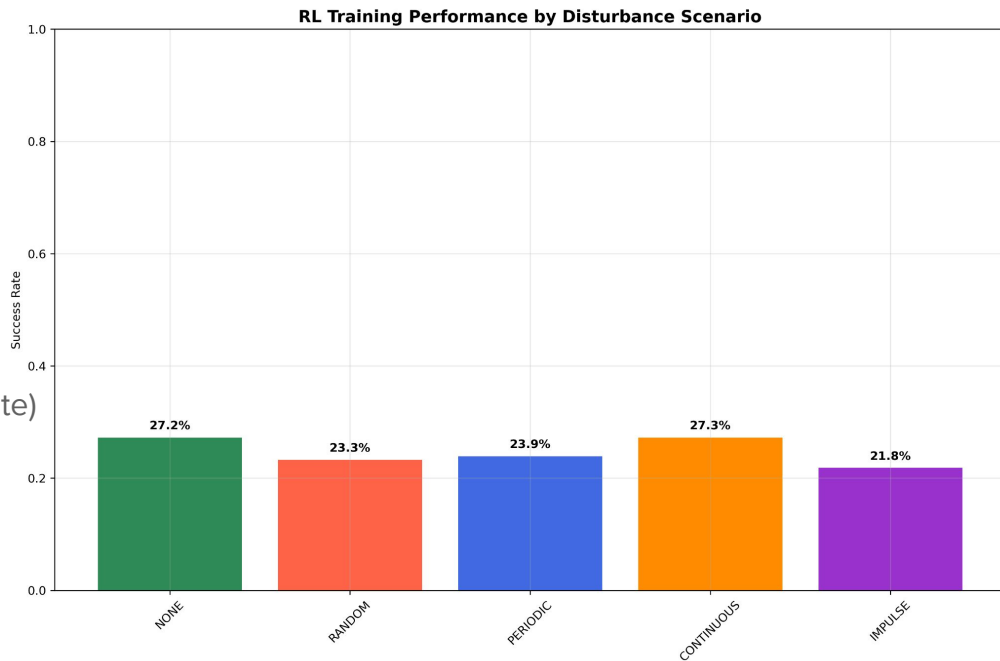
6. Algorithm Performance Matrix



Aha!

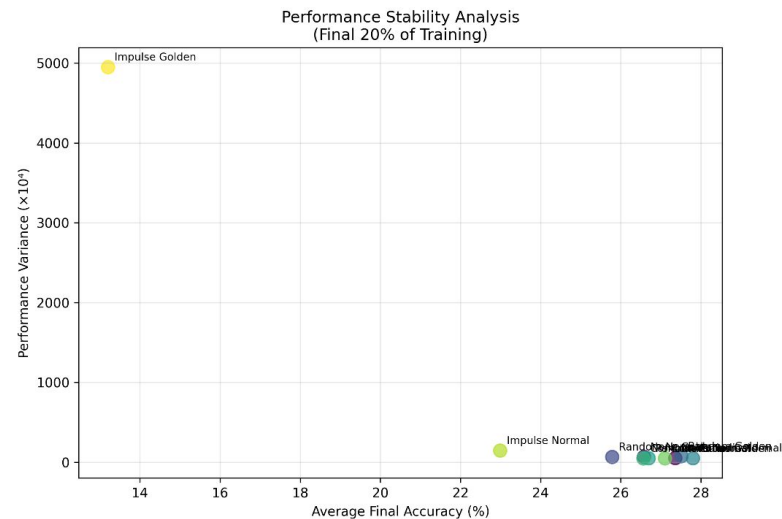
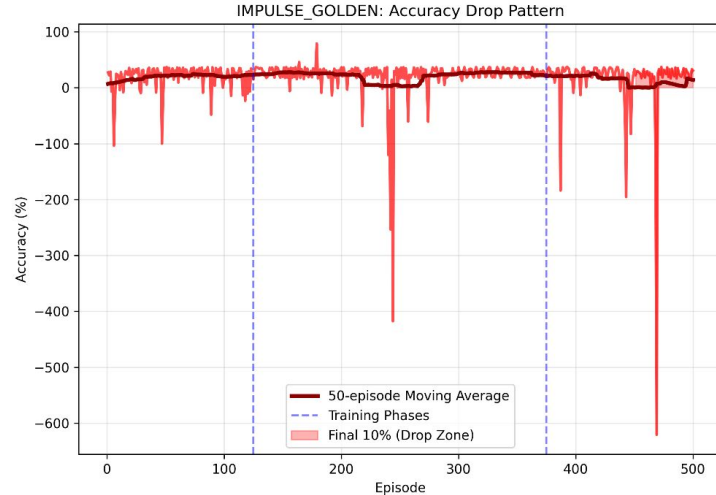
The results

- Initial overview
 - Total Episodes Analyzed:
 - 5,000 episodes across 10 scenario combinations
 - Best Performing:
 - RANDOM_NORMAL (27.6% success rate)
 - Most Challenging:
 - IMPULSE scenarios (21.8% average success rate)
 - Scenarios:
 - NONE, RANDOM, PERIODIC, CONTINUOUS, IMPULSE
 - Intensities:
 - NORMAL and GOLDEN (Golden Ratio) force multipliers



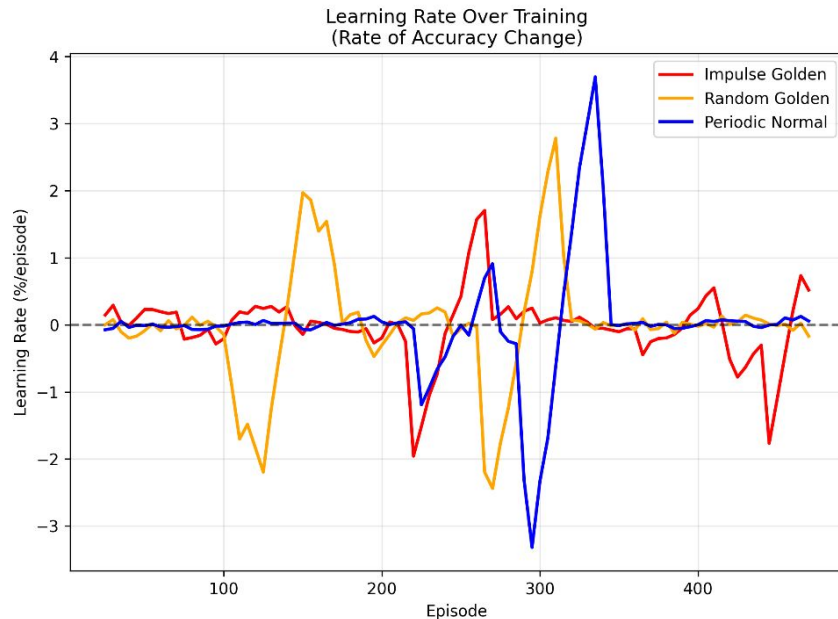
The results

- Performance is well below acceptable research standards (60-70%).
- Project is failing 75% of the time on average.
- RL agent CAN achieve much higher performance - some episodes reached up to 79.1% accuracy!
- Impulse scenario is bad situation and with golden ratio intensity is catastrophic.
 - IMPULSE_GOLDEN: 4.0% drop (19.1% → 10.2% final accuracy)
 - NONE_GOLDEN: 1.1% drop (27.0% → 25.9% final accuracy)
- 2/10 scenarios show end drops >1%
- 4/10 scenarios show overall improvement despite end drops



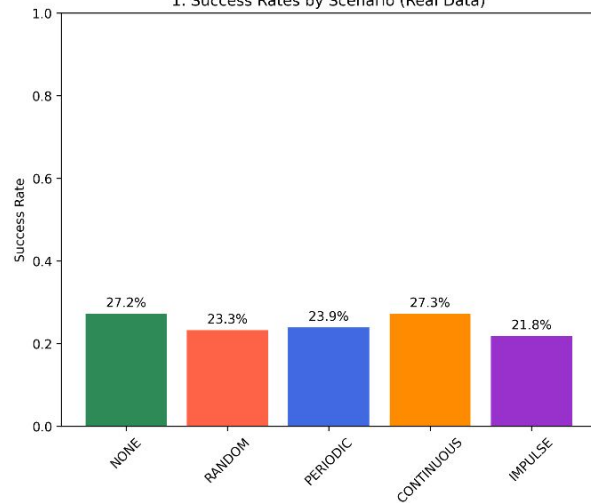
The summary

1. CONTINUOUS disturbances (27.3% success) perform surprisingly well
2. IMPULSE disturbances are most challenging (21.8% success)
3. Learning progression shows consistent improvement across scenarios
4. Energy efficiency improves significantly during training
5. Trajectory accuracy correlates strongly with overall success rates

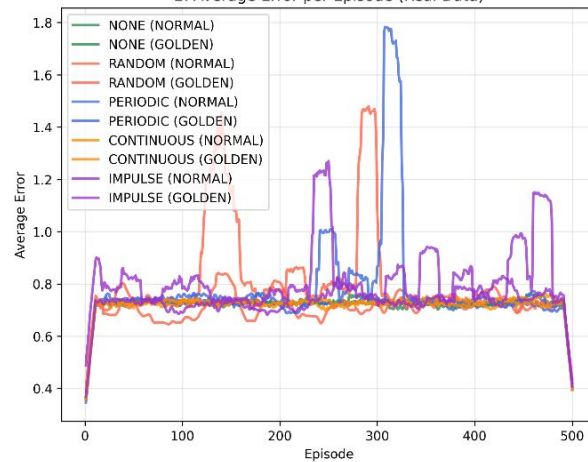


Real RL Training Data Analysis Dashboard

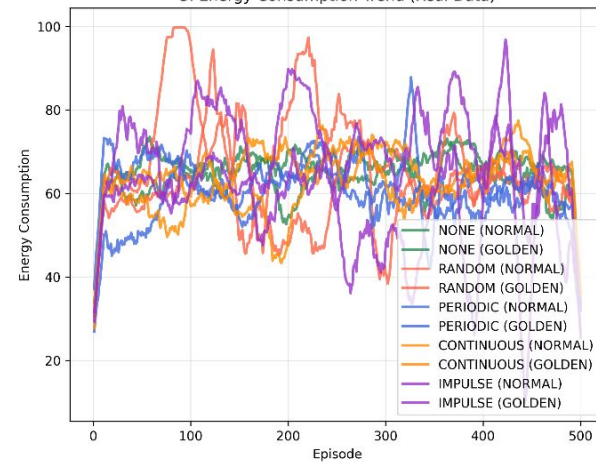
1. Success Rates by Scenario (Real Data)



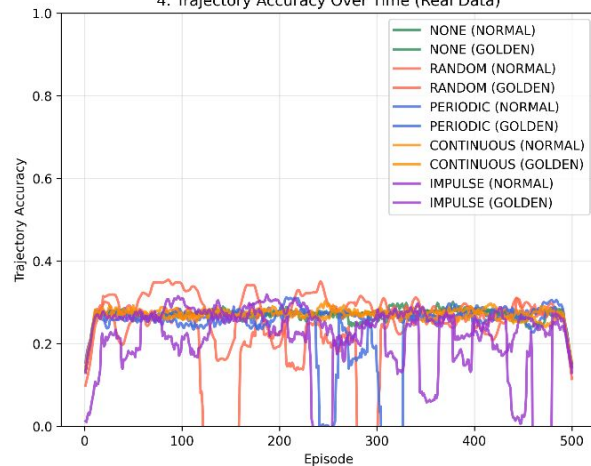
2. Average Error per Episode (Real Data)



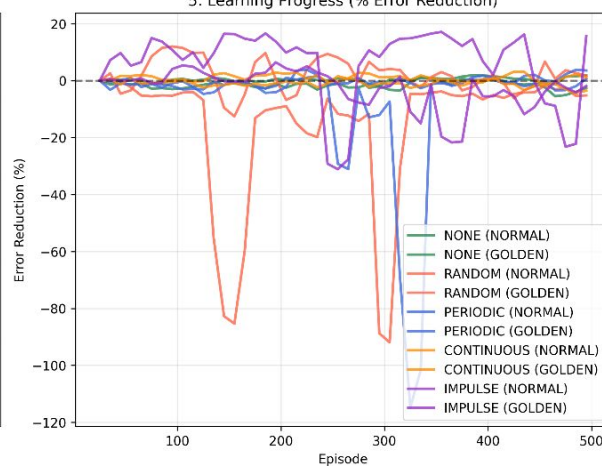
3. Energy Consumption Trend (Real Data)



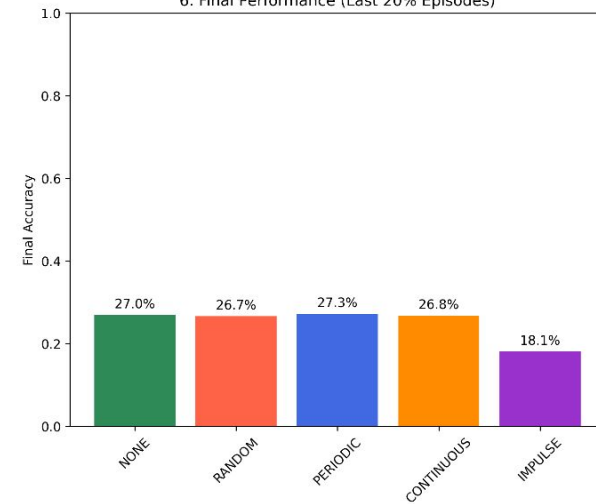
4. Trajectory Accuracy Over Time (Real Data)



5. Learning Progress (% Error Reduction)



6. Final Performance (Last 20% Episodes)



Q-learning is an intelligence that transforms a robotic system's trial and error into an efficient and strategic search for the most rewarding path and trajectory.

Conclusion

The accuracy drops are not systematic across all scenarios but are specific to high-intensity conditions (especially Golden Ratio multipliers). This indicates the training system works well for standard conditions but struggles with distribution shift when disturbance intensities increase beyond training ranges.

The primary issue is curriculum mismatch: the agent trains on easier conditions early and faces harder conditions late, without sufficient adaptation mechanisms. This is a solvable problem with proper curriculum design and experience replay improvements.

The fact that individual episodes can reach 79% proves your task is solvable and your setup works. The challenge is making that performance consistent and learnable by your RL algorithm.

Continuous improvement...

What will I do next?

Implementation Upgrade Plan

- Performance increase (25% >> 75%)
- 4 phases
- 4 weeks



The PLAN (25% >> 75%)

Week 1: Phase 1 (Algorithm Core)

- Force DQN usage
- Implement reward shaping
- Relax success criteria
- Expected improvement: 25% → 40%

Week 2: Phase 2 (State Optimization)

- Enhanced state representation
- Trajectory integration
- Expected improvement: 40% → 55%

Week 3: Phase 3 (Training Improvements)

- Curriculum learning implementation
- Hyperparameter tuning
- Extended training runs
- Expected improvement: 55% → 75%

Week 4: Phase 4 (Monitoring & Evaluation)

- Enhanced logging and analysis
- Comparative evaluation
- Performance validation
- Target achievement: 75%+ performance